

TAREA PPS01

Evaluación de software

Autor: Carlos España Del Pozo - CETI

Contenido

Detalles de la tarea de esta unidad	3
Apartado 1: Configurar el entorno	4
Respuestas 1	4
Apartado 2: Rellenar Tabla.....	11
Respuestas 2	11
Apartado 3: Crear un programa PHP que genere la secuencia de Fibonacci.	12
Respuestas 3	12
Apartado 4: Creación del programa de pruebas unitarias.....	15
Respuestas 4	16
Apartado 5: Verificación de software	17
Respuestas 5	18
Apartado 6: Responde a la pregunta	19
Respuestas 6	20
Bibliografía	20

Detalles de la tarea de esta unidad

Caso práctico

La mayoría de los lenguajes de programación disponen de librerías que simplifican el proceso de creación y ejecución de las pruebas unitarias o de examen de unidad. Por ejemplo, en PHP está PHPUnit, en Java JUnit y en Python unittest. Los pasos a seguir para crear el programa de pruebas suelen ser siempre los mismos:

- Importar la librería de test de software (PHPUnit en PHP)
- Definir una clase donde incluiría la función para comprobar esta parte específica del código (normalmente el nombre debe comenzar con test).
- Realizar los casos de prueba necesarios (para los cuales se suele utilizar el método assertEquals)

Julián ha empezado a trabajar con un nuevo equipo de desarrollo que trabaja con Visual Studio Code y PHP como lenguaje. Ha instalado Visual Studio Code, PHP y ha configurado el editor/IDE para poder trabajar de la mejor forma posible con PHP, incluyendo poder realizar las pruebas unitarias (ha sido necesario instalar PHPUnit).

Para comprobar que todo esta bien ha creado el siguiente programa php que ha llamado **ejemplo.php**:

```
<?php
function obtenerPrimerosNumeros($n) {
    $numeros = array();
    for ($i = 1; $i <= $n; $i++) {
        $numeros[] = $i;
    }
    return $numeros;
}
?>
```

Y para realizar las pruebas ha creado el siguiente programa php que ha llamado **ejemploTest.php**:

LINEA	CODIGO
1	<?php
2	require 'ejemplo.php';
3	// Importa el archivo con la función a probar
4	use PHPUnit\Framework\TestCase;
5	// utiliza la librería con PHPUnit

6	class EjemploTest extends TestCase {
7	// crea una clase que hereda de TestCase
8	public function testObtenerPrimerosNumeros() {
9	// crea un método para realizar las pruebas
10	\$n = 5;
11	\$resultado_esperado = [1, 2, 3, 4, 5];
12	\$resultado_obtenido = obtenerPrimerosNumeros(\$n);
13	\$this->assertEquals(\$resultado_esperado, \$resultado_obtenido);
14	// comprueba que el resultado esperado es igual al obtenido
15	}
16	}
17	?>

Ha ejecutado las pruebas unitarias y el resultado es correcto así que ya está preparado para empezar a trabajar en el nuevo proyecto.

En el nuevo equipo le han pedido desarrollar un pequeño programa que **devuelve la serie de Fibonacci hasta un número dado**. Como es lógico, debe hacer un test del software de tipo examen de unidad o prueba unitaria, es decir quiere verificar que una parte del código se comporta de forma esperada.

Julián ha pensado que como caso de prueba comprobará si el quinto número de la serie Fibonacci es el número 3, en cuyo caso es correcto (la serie Fibonacci es 0,1,1,2,3,5,8,13...).

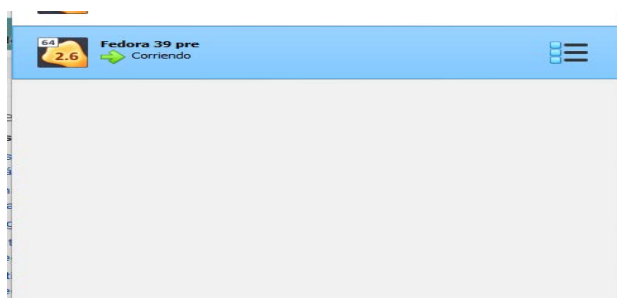
Apartado 1: Configurar el entorno

- Instalar PHP junto con la librería PHPUnit
- Instalar Visual Studio Code y configurar el entorno para poder trabajar con PHP y poder realizar las pruebas unitarias en el mismo entorno

Respuestas 1

Vamos a instalar el entorno, para ello, prepararemos el entorno: utilizaremos:

- **Máquina Virtual Virtual Box con Fedora 39**



INSTALACIÓN DE PHP

Como root, vemos las versiones disponibles:

yum list available php

```

root@fedora:~
Archivo Editar Ver Buscar Terminal Ayuda
root@fedora:~# yum list available php
Última comprobación de caducidad de metadatos hecha hace 0:14:47, el lun 10 nov 2025 16:29:30.
Paquetes disponibles
php.x86_64                                8.2.25-1.fc39                                updates
root@fedora:~#

```

- A continuación, instalaremos desde los repositorios de fedora los paquetes de php8.2

yum install php.x86_64

```

root@fedora:~ -- /usr/bin/python3 /usr/bin/yum install php.x86_64
Archivo Editar Ver Buscar Terminal Ayuda
root@fedora:~# yum install php.x86_64
Última comprobación de caducidad de metadatos hecha hace 0:16:51, el lun 10 nov 2025 16:29:30.
Dependencias resueltas.
=====
Paquete                Arquitectura  Versión              Repositorio  Tam.
-----
Instalando:
php                    x86_64        8.2.25-1.fc39        updates      12 k
Instalando dependencias:
apr                    x86_64        1.7.5-1.fc39         updates      127 k
apr-util               x86_64        1.6.3-4.fc39         fedora       96 k
fedora-logos-httpd     noarch       38.1.0-2.fc39        fedora       15 k
httpd-core             x86_64        2.4.62-2.fc39        updates      1.4 M
httpd-filesystem       noarch       2.4.62-2.fc39        updates      12 k
httpd-tools            x86_64        2.4.62-2.fc39        updates      80 k
nginx-filesystem       noarch       2.1.26.2-1.fc39      updates      11 k
php-common             x86_64        8.2.25-1.fc39        updates      866 k
Instalando dependencias débiles:
apr-util-bdb           x86_64        1.6.3-4.fc39         fedora       13 k
apr-util-openssl       x86_64        1.6.3-4.fc39         fedora       15 k
httpd                  x86_64        2.4.62-2.fc39        updates      46 k
julietaula-montserrat-fonts noarch     1:7.222-8.fc39       updates      1.6 M
mod_http2              x86_64        2.0.29-1.fc39        updates      162 k
mod_lua                x86_64        2.4.62-2.fc39        updates      57 k
php-cli                x86_64        8.2.25-1.fc39        updates      5.4 M
php-fpm                x86_64        8.2.25-1.fc39        updates      1.9 M
php-mbstring           x86_64        8.2.25-1.fc39        updates      575 k
php-opcache            x86_64        8.2.25-1.fc39        updates      619 k
php-pdo                x86_64        8.2.25-1.fc39        updates      118 k

```

Vemos la versión instalada

php -v

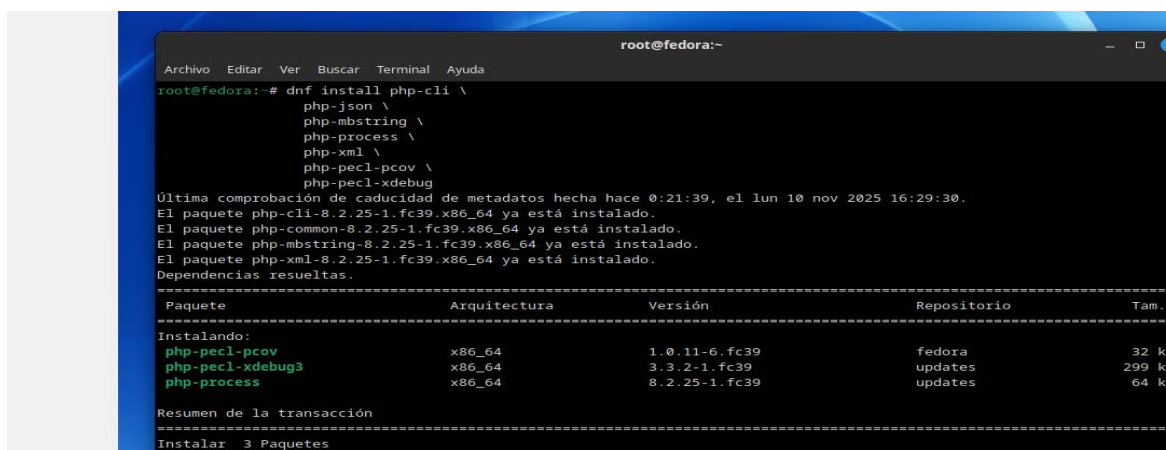
```

root@fedora:~
Archivo Editar Ver Buscar Terminal Ayuda
root@fedora:~# php -v
PHP 8.2.25 (cli) (built: Oct 22 2024 15:12:03) (NTS gcc x86_64)
Copyright (c) The PHP Group
Zend Engine v4.2.25, Copyright (c) Zend Technologies
with Zend OPcache v8.2.25, Copyright (c), by Zend Technologies

```

- Instalamos módulos necesarios para PHPUnit.

```
dnf install php-cli \
php-json \
php-mbstring \
php-process \
php-xml \
php-pecl-pcov \
php-pecl-xdebug
```



```

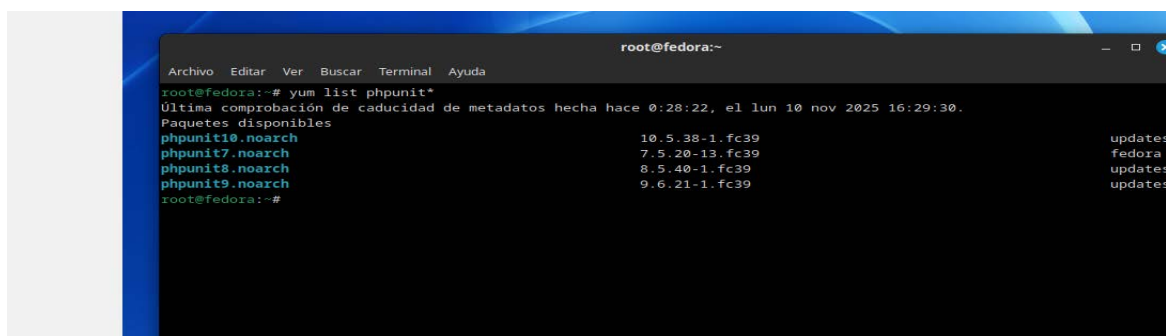
root@fedora:~
Archivo Editar Ver Buscar Terminal Ayuda
root@fedora:~# dnf install php-cli \
php-json \
php-mbstring \
php-process \
php-xml \
php-pecl-pcov \
php-pecl-xdebug
Última comprobación de caducidad de metadatos hecha hace 0:21:39, el lun 10 nov 2025 16:29:30.
El paquete php-cli-8.2.25-1.fc39.x86_64 ya está instalado.
El paquete php-common-8.2.25-1.fc39.x86_64 ya está instalado.
El paquete php-mbstring-8.2.25-1.fc39.x86_64 ya está instalado.
El paquete php-xml-8.2.25-1.fc39.x86_64 ya está instalado.
Dependencias resueltas.
=====
Paquete                Arquitectura      Versión           Repositorio      Tam.
=====
Instalando:
php-pecl-pcov           x86_64            1.0.11-6.fc39     fedora            32 k
php-pecl-xdebug3        x86_64            3.3.2-1.fc39     updates          299 k
php-process             x86_64            8.2.25-1.fc39     updates           64 k
=====
Resumen de la transacción
=====
Instalar 3 Paquetes

```

INSTALACIÓN DE PHPUnit

Como root, vemos las versiones disponibles:

```
yum list available php
```



```

root@fedora:~
Archivo Editar Ver Buscar Terminal Ayuda
root@fedora:~# yum list phpunit*
Última comprobación de caducidad de metadatos hecha hace 0:28:22, el lun 10 nov 2025 16:29:30.
Paquetes disponibles
phpunit10.noarch      10.5.38-1.fc39     updates
phpunit7.noarch       7.5.20-13.fc39     fedora
phpunit8.noarch       8.5.40-1.fc39     updates
phpunit9.noarch       9.6.21-1.fc39     updates
root@fedora:~#

```

- A continuación, instalaremos desde los repositorios de fedora los paquetes de phpunit10.noarch

```
yum install phpunit10.noarch
```

```

root@fedora:~ -- /usr/bin/python3 /usr/bin/yum install phpunit10.noarch

root@fedora:~# yum install phpunit10.noarch
Última comprobación de caducidad de metadatos hecha hace 0:29:23, el lun 10 nov 2025 16:29:30.
Dependencias resueltas.
=====
Paquete                                Arquitectura  Versión      Repositorio  Tam.
-----
Instalando:
phpunit10                              noarch       10.5.38-1.fc39 updates      490 k
Instalando dependencias:
php-fedora-autoloader                  noarch       1.0.1-11.fc39 fedora       12 k
php-myclabs-deep-copy                  noarch       1.12.1-1.fc39 updates     27 k
php-nikic-php-parser5                  noarch       5.1.0-1.fc39 updates    210 k
php-phar-io-manifest2                  noarch       2.0.4-1.fc39 updates     35 k
php-phar-io-version3                  noarch       3.2.1-4.fc39 fedora       22 k
php-phpunit-php-code-coverage10        noarch       10.1.15-1.fc39 updates    244 k
php-phpunit-php-file-iterator4         noarch       4.1.0-1.fc39 fedora       14 k
php-phpunit-php-invoker4               noarch       4.0.0-2.fc39 fedora       12 k
php-phpunit-php-text-template3         noarch       3.0.1-1.fc39 fedora       13 k
php-phpunit-php-timer6                 noarch       6.0.0-2.fc39 fedora       12 k
php-sebastian-cli-parser2              noarch       2.0.0-2.fc39 fedora       14 k
php-sebastian-code-unit-reverse-lookup3 noarch       3.0.0-2.fc39 fedora       12 k
php-sebastian-code-unit2               noarch       2.0.0-2.fc39 fedora       20 k
php-sebastian-comparator5              noarch       5.0.3-1.fc39 updates     18 k
php-sebastian-complexity3              noarch       3.2.0-1.fc39 updates     16 k
php-sebastian-diff5                    noarch       5.1.0-1.fc39 updates     28 k
php-sebastian-environment6             noarch       6.1.0-1.fc39 updates     14 k
php-sebastian-exporter5                noarch       5.1.1-1.fc39 updates     14 k
php-sebastian-global-state6            noarch       6.0.1-2.fc39 fedora       15 k
php-sebastian-lines-of-code2           noarch       2.0.2-1.fc39 updates     15 k

```

Vemos la versión instalada

phpunit --version

```

root@fedora:~# phpunit --version
PHPUnit 10.5.38 by Sebastian Bergmann and contributors.

root@fedora:~#

```

INSTALACIÓN DE VISUAL STUDIO CODE

Como root, instalamos los repositorios de "Microsoft"

```

sudo rpm --import https://packages.microsoft.com/keys/microsoft.asc
echo -e "[code]\nname=Visual Studio
Code\nbaseurl=https://packages.microsoft.com/yumrepos/vscode\nenabled=1\nautorefresh=1\ntype
=rpm-md\nngpgcheck=1\nngpgkey=https://packages.microsoft.com/keys/microsoft.asc" | sudo tee
/etc/yum.repos.d/vscode.repo > /dev/null

```

```

root@fedora:~# sudo rpm --import https://packages.microsoft.com/keys/microsoft.asc
echo -e "[code]\nname=Visual Studio Code\nbaseurl=https://packages.microsoft.com/yumrepos/vscode\nenabled=1\nautorefresh=1\ntype=rpm-md\nngpgcheck=1\nngpgkey=https://packages.microsoft.com/keys/microsoft.asc" | sudo tee /etc/yum.repos.d/vscode.repo > /dev/null
root@fedora:~#

```


- A continuación, instalaremos desde los repositorios de microsoft los paquetes de code
- yum install code**

```

root@fedora:~# yum install code
Visual Studio Code
Última comprobación de caducidad de metadatos hecha hace 0:00:02, el lun 10 nov 2025 17:05:13.
Dependencias resueltas.
=====
Paquete      Arquitectura  Versión      Repositorio  Tam.
-----
Instalando:
code         x86_64        1.105.1-1760482588.el8      code         155 M

Resumen de la transacción
=====
Instalar 1 Paquete

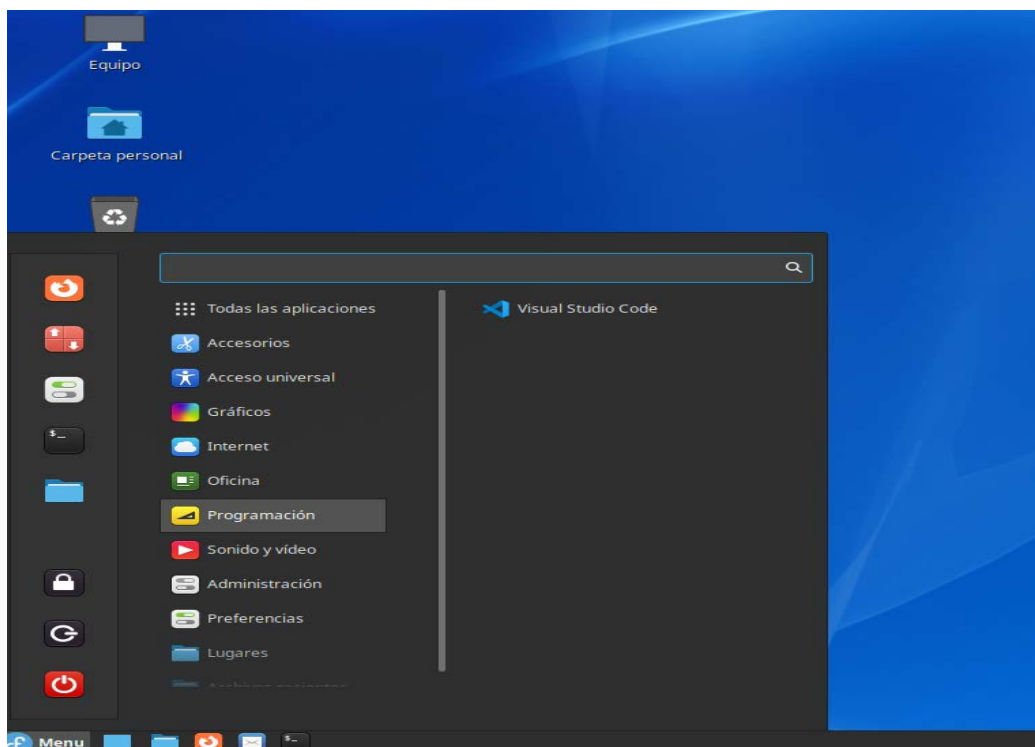
Tamaño total de la descarga: 155 M
Tamaño instalado: 435 M
¿Está de acuerdo [s/N]? : s
Descargando paquetes:
code-1.105.1-1760482588.el8.x86_64.rpm
-----
Total
-----
Ejecutando verificación de operación
Verificación de operación exitosa.
Ejecutando prueba de operaciones
Prueba de operación exitosa.
Ejecutando operación
Preparando      : code-1.105.1-1760482588.el8.x86_64      1/1
Instalando      : code-1.105.1-1760482588.el8.x86_64      1/1
Ejecutando scriptlet: code-1.105.1-1760482588.el8.x86_64  1/1

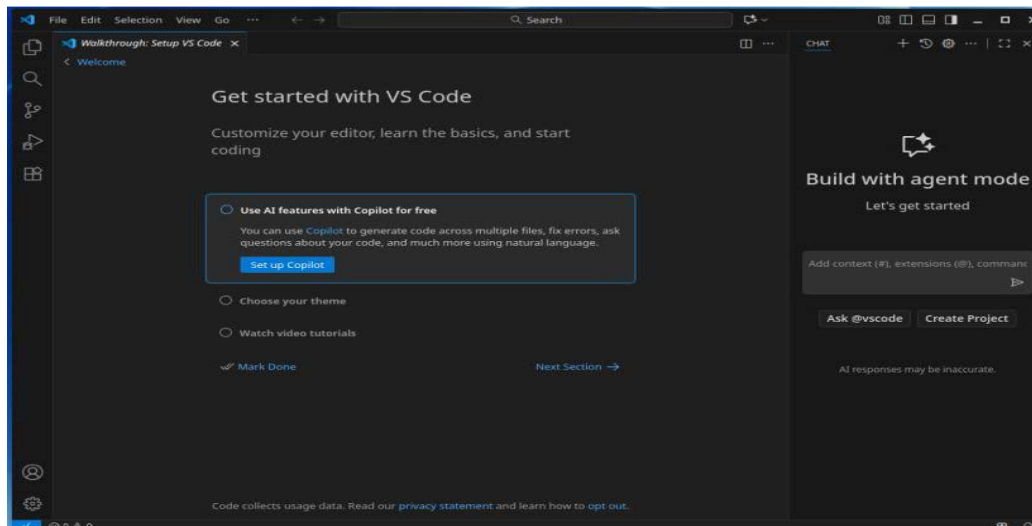
```

Configurar Visual Stdio Code para trabajar con PHP y realizar pruebas unitarias.

- Como usuario “cespana”, abrimos Visual Studio Code

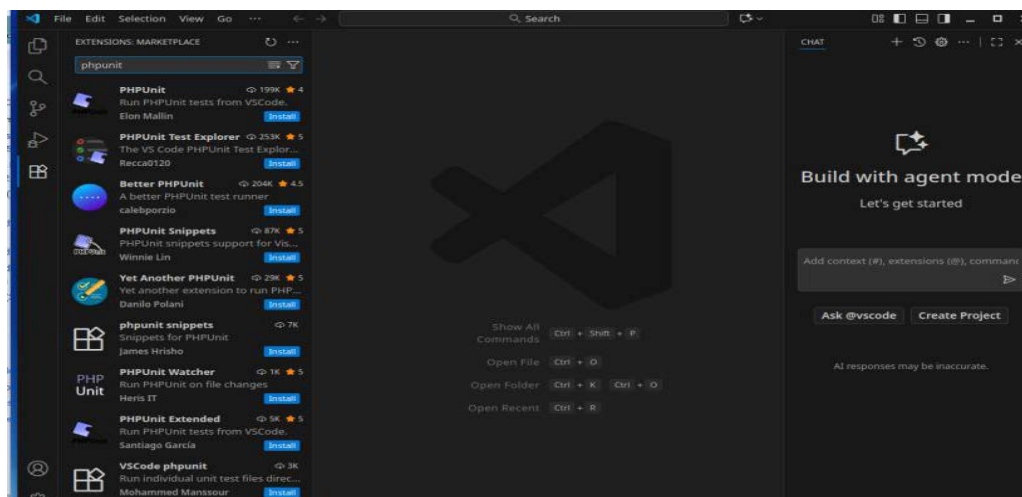
Dentro de menú -> programación -> Visual Studio Code



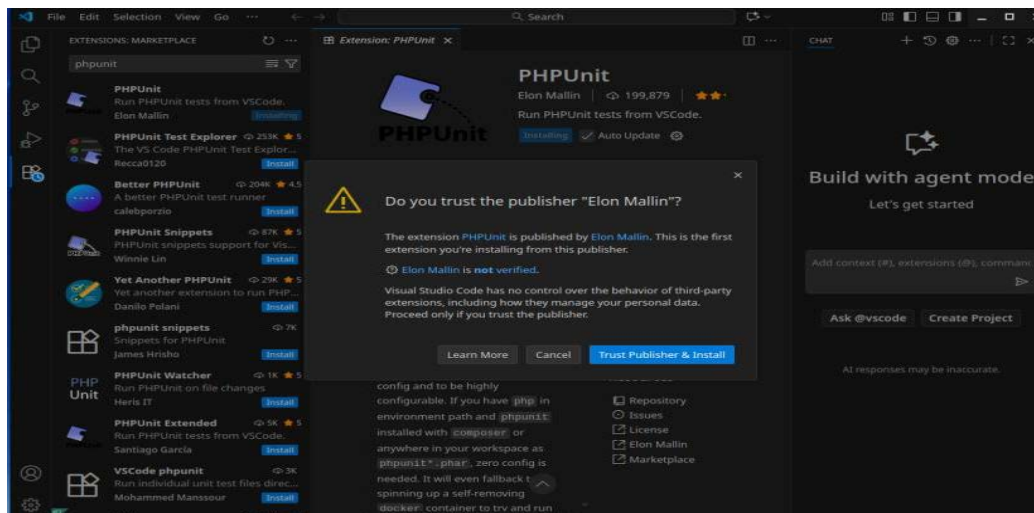


- En la barra lateral izquierda de vsc seleccionaremos extensiones y filtraremos por “phpunit”

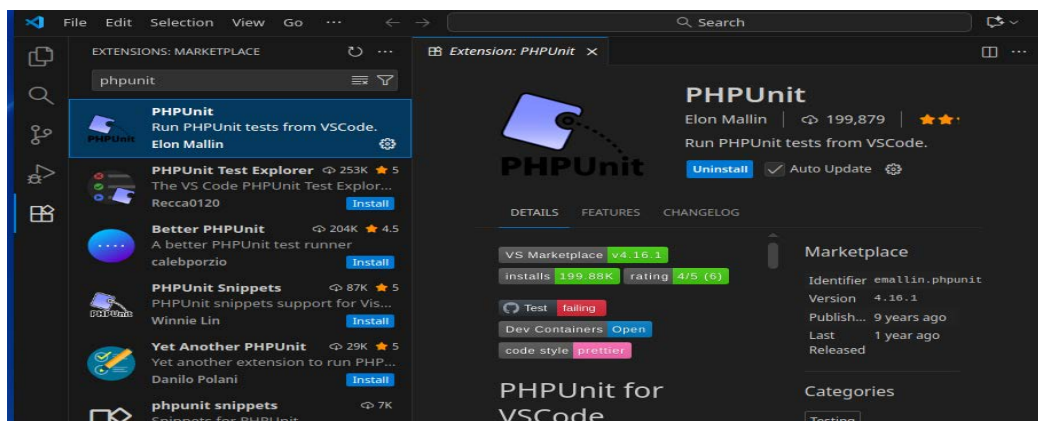
Una vez nos aparezcan pincharemos en “Install” de PHPUnit



Pinchamos en “Trust Publisher & Install



Veremos que aparece como Instalada



Una vez finalizado ya tendríamos Instalado todo el entorno necesario para realizar las pruebas.

Apartado 2: Rellenar Tabla

Rellenar la siguiente tabla utilizando el código del programa de pruebas que se ha dado como ejemplo (ejemploTest.php)

Respuestas 2

PREGUNTA	RESPUESTA
Indica el número de las líneas donde hay un comentario	Las líneas donde vemos comentarios “//” son: <pre> 3 // Importa el archivo con la función a probar 5 // utiliza la librería con PHPUnit 7 // crea una clase que hereda de TestCase 9 // crea un método para realizar las pruebas 14 // comprueba que el resultado esperado es igual al obtenido </pre>
Indica el número de las líneas donde hay comandos de pre-procesamiento	Los comandos de preprocesamiento serían en este caso, “require” y “use” <pre> 2 require 'ejemplo.php'; 4 use PHPUnit\Framework\TestCase; </pre>
Indica el nombre de las variables utilizadas	Las variables declaradas, están precedidas de “\$” <pre> 10 \$n 11 \$resultado_esperado 12 \$resultado_obtenido </pre>
Indica el nombre de los métodos/funciones utilizados	La función para probar “ObtenerPrimerosNumeros(\$n)” y el método de PHPUnit “assertEquals” <pre> 12 obtenerPrimerosNumeros(\$n); 13 assertEquals(\$resultado_esperado, \$resultado_obtenido); </pre>
Indica qué hace las línea 13	Usa el método “assertEquals” de PHPUnit para comparar el resultado esperado con el resultado que devuelve la función. <pre> 13 \$this->assertEquals(\$resultado_esperado, \$resultado_obtenido); </pre>

Apartado 3: Crear un programa PHP que genere la secuencia de Fibonacci.

Se pide crear un pequeño programa PHP para generar la secuencia de Fibonacci. Se ha escogido PHP porque se va a utilizar en las unidades 2 y 3 y además, es uno de los lenguajes más utilizados en las aplicaciones web

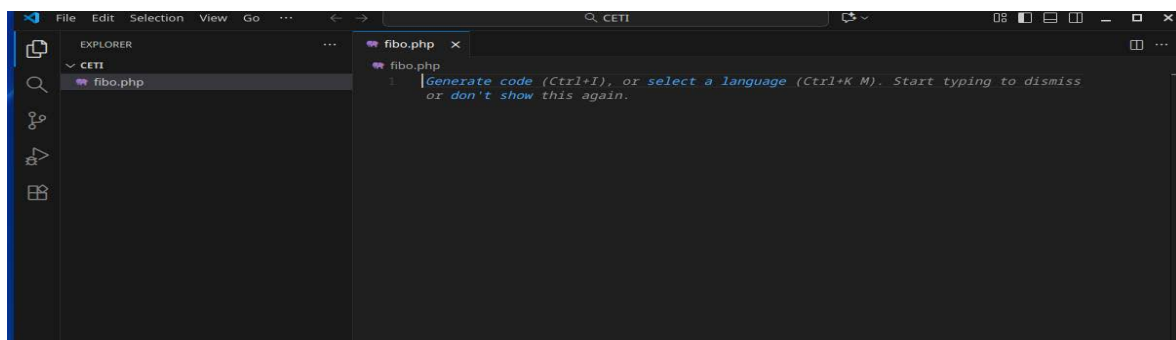
Para realizar la tarea los pasos serán:

- Crearemos el archivo **fibo.php**
- Debes escribir un código donde incluirás:
 - Una línea de comentario donde escribirás tu nombre.
 - Otros comentarios en las líneas que creas importantes
 - Utilizar variables
 - Utilizar alguna estructura de control
 - Utilizar el control de errores (mira este enlace <https://www.php.net/manual/en/language.exceptions.php>).
- Dentro de este archivo crearemos una **funcion llamada fibonacci** que devolverá la secuencia de fibonacci hasta el valor indicado como parámetro. Por ejemplo, fibonacci(6) devolverá [0,1,1,2,3,5] y fibonacci(8) devolverá [0,1,1,2,3,5,8]

Respuestas 3

A continuación, detallo el proceso de creación del programa:

Dentro de Visual Studio code creamos un nuevo fichero en la sección de explorer para escribir el código solicitado.



Crearemos el código solicitado en el ejercicio:

```
<?php
// Carlos España - PPS01 25-26
// Programa secuencia de Fibonacci
```

```
// Función Fibonacci con control de errores
function fibonacci($n) {

    // Comprobamos que el valor sea correcto
    if(!is_numeric($n) || intval($n) != $n || $n < 0){
        throw new Exception("Error: introduce un número entero mayor o
igual que 0.");
    }

    // Valores iniciales de la serie
    $n = intval($n);
    $serie = [0,1];

    // Si el número es 0 o 1, devolvemos directamente esa parte
    if($n <= 1){
        return array_slice($serie, 0, $n+1);
    }

    // Calculamos el resto de la serie
    for($i = 2; $i <= $n; $i++){
        $serie[$i] = $serie[$i-1] + $serie[$i-2];
    }

    return $serie;
}

// Pedimos el número al usuario
echo "Introduce un número para generar la serie de Fibonacci: ";
$entrada = trim(fgets(STDIN));

// Control de errores
try {
    $resultado = fibonacci($entrada);
    echo "Resultado: ";
    print_r($resultado);
} catch (Exception $e){
    echo $e->getMessage();
}
?>
```

```

1  <?php
2  // Carlos España - PP501 25-26
3  // Programa secuencia de Fibonacci
4
5  // Función Fibonacci con control de errores
6  function fibonacci($n) {
7
8      // Comprobamos que el valor sea correcto
9      if(!is_numeric($n) || intval($n) != $n || $n < 0){
10         throw new Exception("Error: introduce un número entero mayor o igual que 0.");
11     }
12
13     // Valores iniciales de la serie
14     $n = intval($n);
15     $serie = [0,1];
16
17     // Si el número es 0 o 1, devolvemos directamente esa parte
18     if($n <= 1){
19         return array_slice($serie, 0, $n+1);
20     }
21
22     // Calculamos el resto de la serie
23     for($i = 2; $i <= $n; $i++){
24         $serie[$i] = $serie[$i-1] + $serie[$i-2];
25     }
26
27     return $serie;
28 }
29
30 // Pedimos el número al usuario
31 echo "Introduce un número para generar la serie de Fibonacci: ";
32 $entrada = trim(fgets(STDIN));
33
34 // Control de errores
35 try {
36     $resultado = fibonacci($entrada);
37     echo "Resultado: ";
38     print_r($resultado);
39 } catch (Exception $e){
40     echo $e->getMessage();
41 }
42 ?>

```

Probamos la funcionalidad del programa:

```

cespana@fedora:~/CETI
Archivo  Editar  Ver  Buscar  Terminal  Ayuda

[1] => 1
[2] => 1
[3] => 2
[4] => 3
[5] => 5
[6] => 8
)
cespana@fedora:~/CETI$ php fibo.php
Introduce un número para generar la serie de Fibonacci: 8
Resultado: Array
(
    [0] => 0
    [1] => 1
    [2] => 1
    [3] => 2
    [4] => 3
    [5] => 5
    [6] => 8
    [7] => 13
    [8] => 21
)
cespana@fedora:~/CETI$ php fibo.php
Introduce un número para generar la serie de Fibonacci: 0,5
Error: introduce un número entero mayor o igual que 0.cespana@fedora:~/CETI$ php
fibo.php

```

Vemos que funciona correctamente y también tiene el control de errores.

Apartado 4: Creación del programa de pruebas unitarias

- Crearemos un programa principal donde definiremos una clase llamada Test para probar nuestro código (tienes un ejemplo en el enunciado de la tarea)

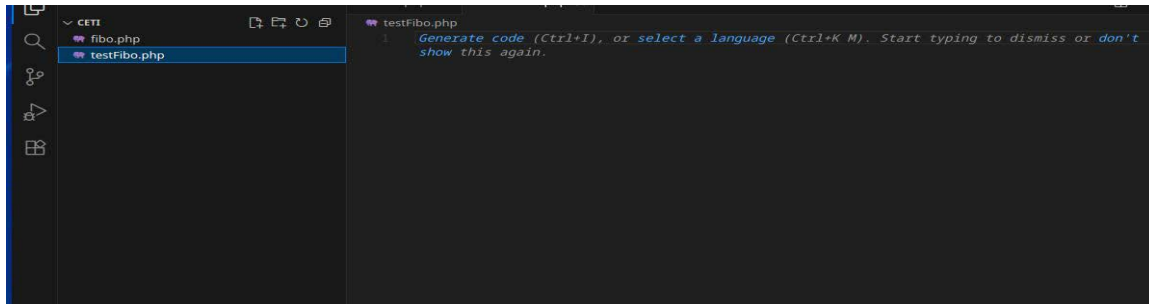
Para realizar la tarea los pasos serán:

- Crearemos un archivo de nombre **testFibo.php**
- Importaremos la librería de testeo de software en este caso PHPUnit.
- Importaremos el archivo donde se ha definido la función fibonacci
- Crearemos nuestra clase (del tipo TestCase)
- Dentro de esta clase definiremos una función (es recomendable un nombre ilustrativo que empiece por test)
- Dentro de la función reflejaremos el tipo de testeo que vamos a realizar. En este caso nuestro objetivo es verificar si la posición X de la sucesión de Fibonacci coincide con el resultado esperado, es decir coincide con la posición X que devuelve nuestro programa. Es decir queremos verificar que el quinto número de la sucesión, que es 3, coincide con el quinto número que devuelve nuestra secuencia.
 - Para hacer esta comprobación usaremos el tipo de comprobación **assertEquals** que comprueba si dos valores son iguales
 - Mediante esta función comprobaremos si la posición quinta de nuestra función coincide con el valor esperado (el valor esperado es la quinta posición de la sucesión que es 3)

Respuestas 4

A continuación, detallo el proceso de creación del programa:

Dentro de Visual Studio code creamos un nuevo fichero en la sección de explorer para escribir el código solicitado.



Crearemos el código solicitado en el ejercicio:

```
<?php
// Carlos España - PPS01 25-26
// Archivo de pruebas para la función Fibonacci

// Importamos el archivo donde está definida la función fibonacci
require 'fibo.php';

// Importamos PHPUnit
use PHPUnit\Framework\TestCase;

// crea una clase que hereda de TestCase
class TestFibo extends TestCase {

    // crea un método para realizar las pruebas
    public function testQuintoNumeroFibonacci() {

        // Valor que queremos comprobar (posición 5)
        $n = 4;

        // Valor esperado en la posición 5 (realmente índice 4)
        $valorEsperado = 3;

        // Ejecutamos nuestra función y obtenemos el array
        $resultado = fibonacci($n);

        // Comprobamos el valor en la posición indicada
        $this->assertEquals($valorEsperado, $resultado[$n]);
    }
}
?>
```

```
testFibo.php
1  <?php
2  // Carlos España - PPS01 25-26
3  // Archivo de pruebas para la función Fibonacci
4
5  // Importamos el archivo donde está definida la función fibonacci
6  require 'fibo.php';
7
8  // Importamos PHPUnit
9  use PHPUnit\Framework\TestCase;
10
11 // crea una clase que hereda de TestCase
12 class TestFibo extends TestCase {
13
14     // crea un método para realizar las pruebas
15     public function testQuintoNumeroFibonacci() {
16
17         // Valor que queremos comprobar (posición 5)
18         $n = 4;
19
20         // Valor esperado en la posición 5 (realmente índice 4)
21         $valorEsperado = 3;
22
23         // Ejecutamos nuestra función y obtenemos el array
24         $resultado = fibonacci($n);
25
26         // Comprobamos el valor en la posición indicada
27         $this->assertEquals($valorEsperado, $resultado[$n]);
28     }
29 }
30 ?>
```

Apartado 5: Verificación de software

Ejecutamos la prueba y comprobamos que no hay error

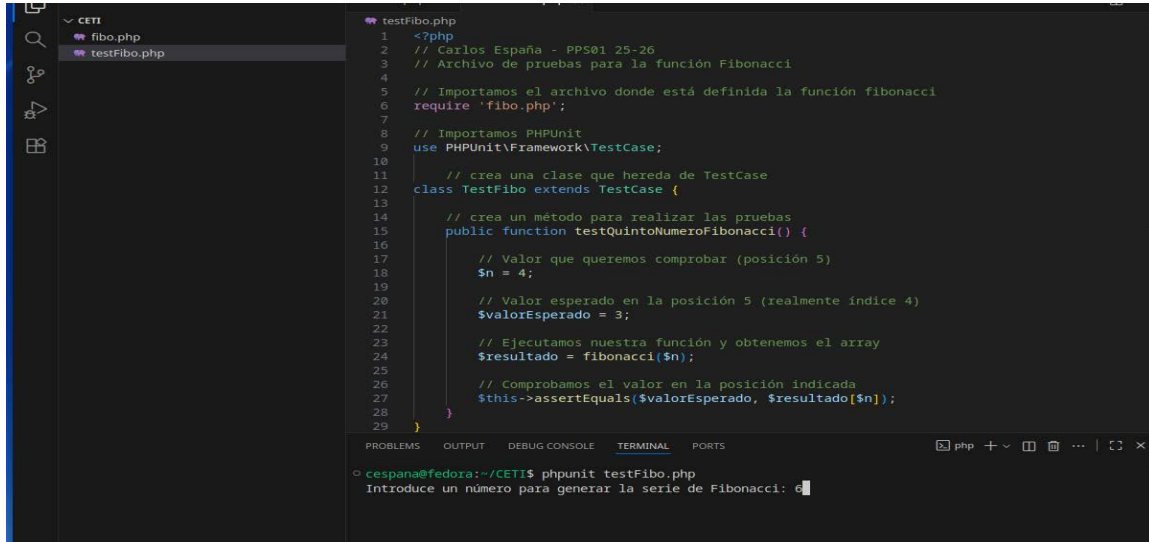
Pasos a realizar:

- Ejecutaremos el programa final de pruebas (testFibo.php) y verificaremos si realmente nuestro programa que calcula la sucesión de Fibonacci se comporta como esperamos.
- Si el programa que comprueba el código detecta un error nos reflejará que dato está esperando y que ha recibido

Respuestas 5

Ejecución de las pruebas:

Abrimos un terminal en VisualStudioCode



```

1  <?php
2  // Carlos España - PPS01 25-26
3  // Archivo de pruebas para la función Fibonacci
4
5  // Importamos el archivo donde está definida la función fibonacci
6  require 'fibo.php';
7
8  // Importamos PHPUnit
9  use PHPUnit\Framework\TestCase;
10
11 // crea una clase que hereda de TestCase
12 class TestFibo extends TestCase {
13
14 // crea un método para realizar las pruebas
15 public function testQuintoNumeroFibonacci() {
16
17 // Valor que queremos comprobar (posición 5)
18 $n = 4;
19
20 // Valor esperado en la posición 5 (realmente índice 4)
21 $valorEsperado = 3;
22
23 // Ejecutamos nuestra función y obtenemos el array
24 $resultado = fibonacci($n);
25
26 // Comprobamos el valor en la posición indicada
27 $this->assertEquals($valorEsperado, $resultado[$n]);
28
29 }

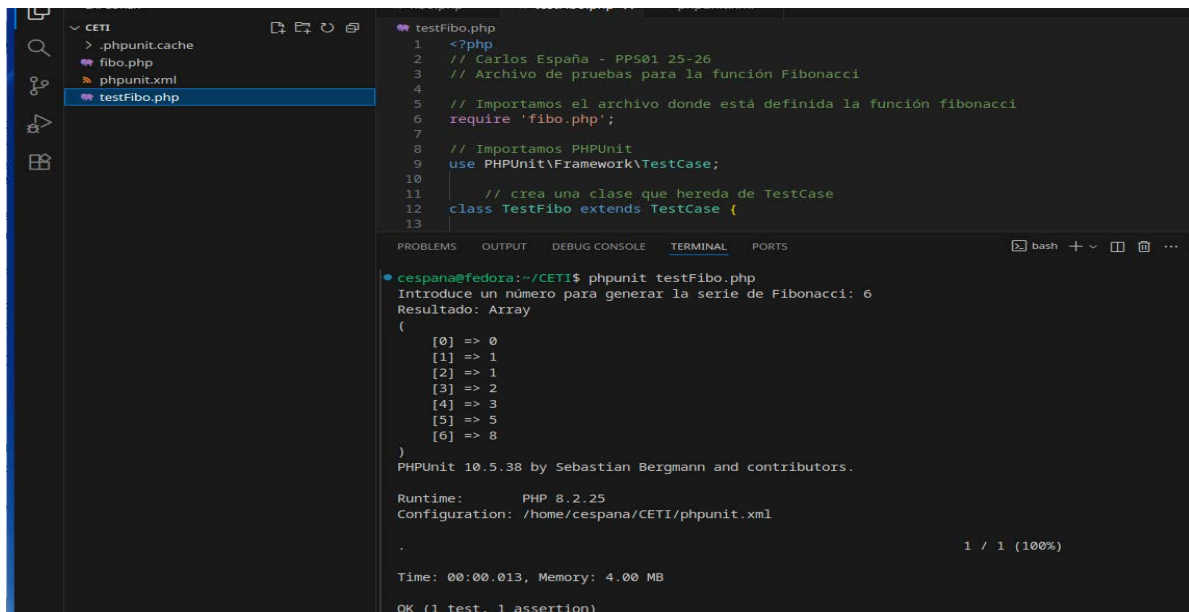
```

```

cespana@fedora:~/CETI$ phpunit testFibo.php
Introduce un número para generar la serie de Fibonacci: 6

```

Comprobamos que nuestro programa “testFibo.php” funciona correctamente



```

1  <?php
2  // Carlos España - PPS01 25-26
3  // Archivo de pruebas para la función Fibonacci
4
5  // Importamos el archivo donde está definida la función fibonacci
6  require 'fibo.php';
7
8  // Importamos PHPUnit
9  use PHPUnit\Framework\TestCase;
10
11 // crea una clase que hereda de TestCase
12 class TestFibo extends TestCase {
13

```

```

cespana@fedora:~/CETI$ phpunit testFibo.php
Introduce un número para generar la serie de Fibonacci: 6
Resultado: Array
(
    [0] => 0
    [1] => 1
    [2] => 1
    [3] => 2
    [4] => 3
    [5] => 5
    [6] => 8
)
PHPUnit 10.5.38 by Sebastian Bergmann and contributors.

Runtime:    PHP 8.2.25
Configuration: /home/cespana/CETI/phpunit.xml

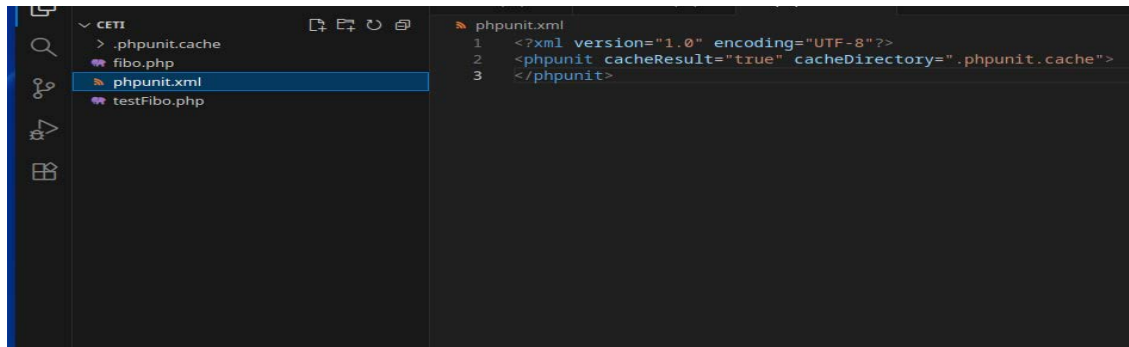
1 / 1 (100%)

Time: 00:00.013, Memory: 4.00 MB

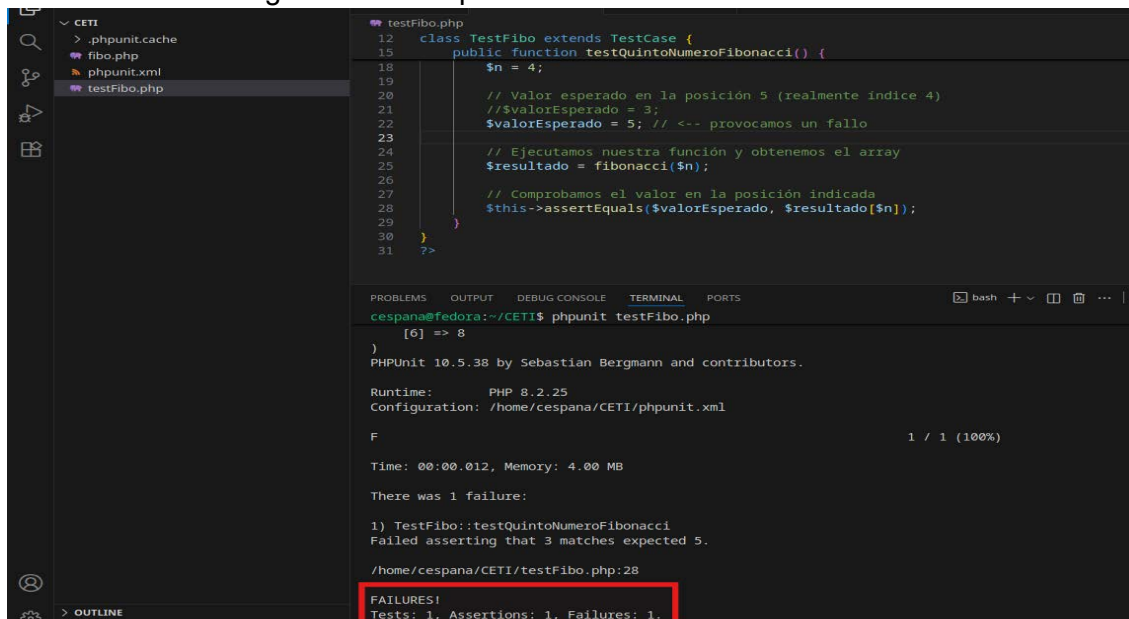
OK (1 test, 1 assertion)

```

NOTA: Hemos creado un archivo “xml” para crear un directorio cache “.phpunit.cache” para el usuario “cespana”



Modificamos el código de testFibo para forzar el fallo:



Apartado 6: Responde a la pregunta

Dado el siguiente código PHP que comprueba la conexión a una base de datos MySQL y que si se ejecuta funciona:

```

1 <?php
2 require_once 'pdoconfig.php';
3 $servername = "db1";
4 $database = "databasename";
5 $username = "username";
6 $password = "password";
7
8 try {
9     $conn = new PDO("mysql:host=$servername;dbname=$database", $username, $password);
10     echo "Conectado a $database de $servername con exito.";
11 } catch (PDOException $pe) {
12     die("No es posible la conexión a $database :". $pe->getMessage());
13 }
14 ?>
    
```

Si realizamos un análisis estático de código nos va a devolver un error del tipo "**Hard-coded credentials are security-sensitive**" ¿Por qué? Razona la respuesta. Indica la(s) línea(s) que habría que cambiar

Respuestas 6

Una vez analizado el código facilitado y que suele ser una práctica, demasiado común en la cadena de conexión de las aplicaciones PHP a BBDD. Podemos asegurar que los siguientes datos:

```
$servername = "db1";  
$database = "databasename";  
$username = "username";  
$password = "password";
```

Están definidos directamente en la BBDD y por lo tanto son visibles si alguna amenaza tiene acceso al código / ficheros.

Lo más razonable sería no pasar los datos directamente y generar variables de entorno:

```
$servername = getenv('DB_SERVER');  
$database = getenv('DB_NAME');  
$username = getenv('DB_USER');  
$password = getenv('DB_PASS');
```

De esta manera **tenemos que definir las en el entorno**, pero sería mucho más seguro.

Bibliografía

<https://phpthrightway.com/>

<https://docs.phpunit.de/en/10.5/installation.html>

https://code.visualstudio.com/docs/setup/linux#_install-vs-code-on-linux

<https://www.php.net/manual/en/language.exceptions.php>